

Лекция 4

ML-пайплайн и оценка качества

kNN, разделение данных и метрики

Лектор

Елизавета Власова

Практики

Кирилл Захаров

Камила Насибуллина

Михаил Подсытник

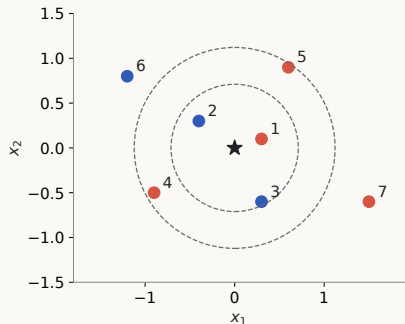
Игорь Толстокулаков

1

kNN как первая модель

От соседей к предсказанию

Класс нового объекта



Звезда — новый объект. Цвет — известный класс обучающих объектов.

Определите класс:

- ▶ при $k = 1$;
- ▶ при $k = 3$;
- ▶ при $k = 5$.

Соседи 1–7 пронумерованы по возрастанию расстояния.

Алгоритм kNN

Сосед	Расстояние	Класс
1	0.316	красный
2	0.500	синий
3	0.671	синий
4	1.030	красный
5	1.082	красный

Для каждого нового объекта:

- ▶ Вычислить расстояния до train.
- ▶ Выбрать k ближайших.
- ▶ Вернуть самый частый класс.

$k = 1$: красный

$k = 3$: синий

$k = 5$: красный

Обучение базового kNN — сохранение примеров (x_i, y_i) ; поиск происходит при предсказании.

ЕВКЛИДОВО И МАНХЭТТЕНСКОЕ РАССТОЯНИЯ

$$d_2(x, u) = \sqrt{\sum_j (x_j - u_j)^2}$$

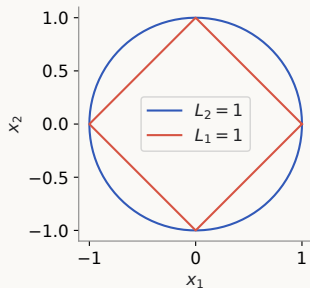
$$d_1(x, u) = \sum_j |x_j - u_j|$$

Для $(0, 0)$ и $(3, 4)$:

$d_2 = 5$, $d_1 = 7$.

На рисунке — все точки на расстоянии 1 от начала координат.

Обобщение: $d_p(x, u) = (\sum_j |x_j - u_j|^p)^{1/p}$, $p \geq 1$.



Косинусная близость



$$\text{sim}_{\cos}(x, u) = \frac{x^T u}{\|x\| \|u\|}, \quad d_{\cos} = 1 - \text{sim}_{\cos}.$$

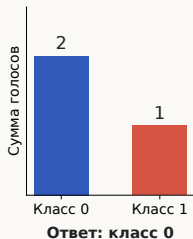
Сравниваем направления ненулевых векторов. Это полезно, например, для поиска по текстовым представлениям.

Ближкий сосед может изменить ответ

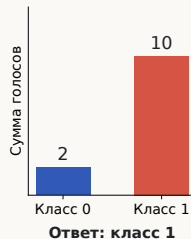
Три ближайших соседа



Один сосед — один голос



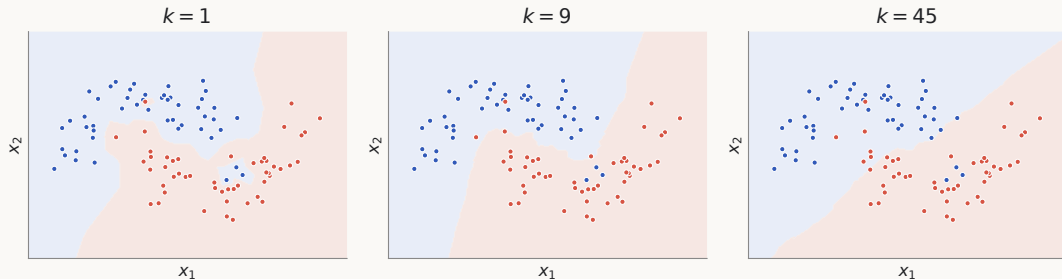
Вес = $1 / \text{расстояние}$



Два далёких соседа голосуют за класс 0. Один близкий — за класс 1.

С весами по расстоянию близкий сосед получает больший голос.

Число соседей и сложность модели



- ▶ Малое k : высокая чувствительность к выборке (variance).
- ▶ Большое k : сильное сглаживание, риск недообучения (bias).

Как выбрать k ? По качеству на данных, не использованных для обучения.

Как задать вес соседа

Соседи упорядочены по расстоянию: $d_1 \leq d_2 \leq \dots \leq d_k$.

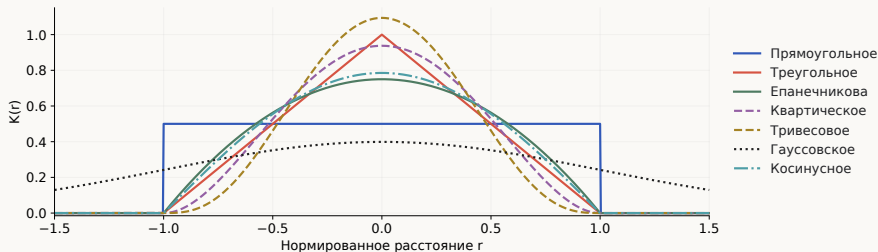
Правило	Формула	Что учитываем
По рангу	$w_i = (k + 1 - i) / k$	Только место в списке
По расстоянию	$w_i = 1 / d_i, d_i > 0$	Насколько сосед близок

Для $k = 3$, $d = (0.1, 1, 1)$ веса будут:

по рангу — $(1, \frac{2}{3}, \frac{1}{3})$; по расстоянию — $(10, 1, 1)$.

Идея: порядок соседей не показывает разницу между расстояниями.

Ядро задаёт вес по расстоянию



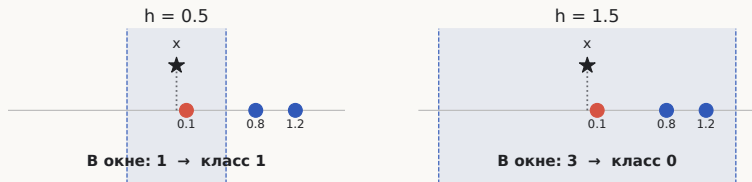
$$w_i = K(r_i), \quad r_i = \frac{d(x, x_i)}{h}, \quad h > 0.$$

Для $r \geq 0$ берём неотрицательные, невозрастающие веса.

У компактных ядер вес равен нулю вне окна; у гауссовского — плавно убывает.

Что означает ширина окна h

Прямоугольное ядро: каждый объект с $d(x, x_i) \leq h$ получает вес 1.



$$\hat{y}(x) = \arg \max_c \sum_{i=1}^n K\left(\frac{d(x, x_i)}{h}\right) 1[y_i = c].$$

Фиксируем h — число объектов в окне меняется.

Фиксируем k — радиус до k -го соседа зависит от положения запроса.

Регрессия: соседи предсказывают число

Вместо голосования по классам усредняем значения y_i .

$$\hat{y}(x) = \frac{\sum_{i \in \mathcal{N}(x)} w_i y_i}{\sum_{i \in \mathcal{N}(x)} w_i}.$$

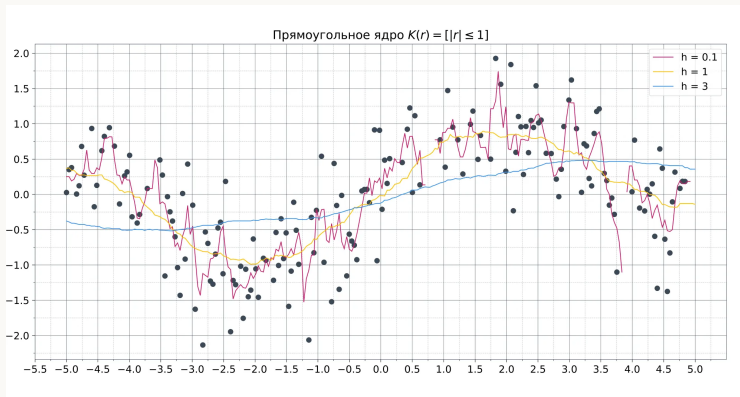
Сосед	1	2	3
Значение y_i	10	20	30
Расстояние d_i	0.1	1	1

Равные веса: $\hat{y} = (10 + 20 + 30)/3 = 20$.

Весы $1/d_i$: $\hat{y} = (10 \cdot 10 + 20 + 30)/12 = 12.5$.

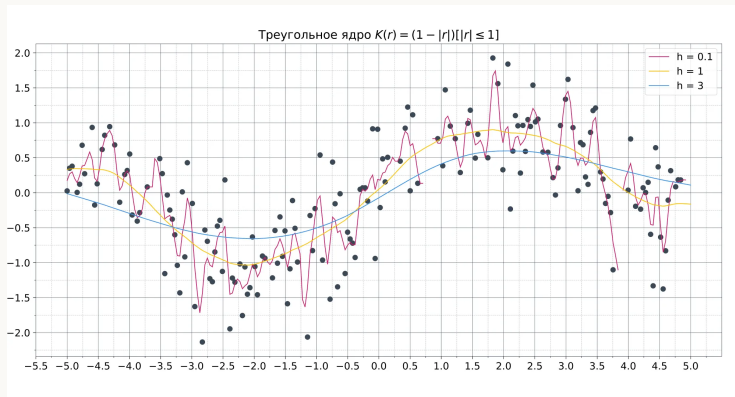
kNN: $\mathcal{N}(x)$ — k соседей. Ядерная регрессия: весь train, $w_i = K(d_i/h)$.

Окно: прямоугольное ядро



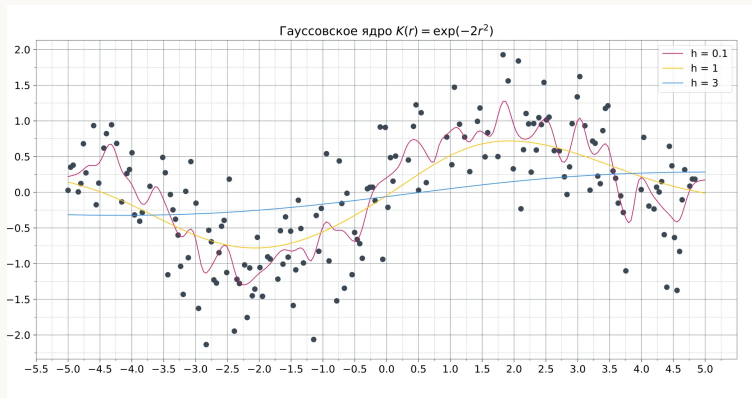
Малое h — резкие локальные изменения; большое h — сильное сглаживание.

Окно: треугольное ядро



Вес плавно падает до нуля на границе окна. Сравните кривые при тех же h .

Окно: гауссовское ядро



Нет жёсткой границы окна. h задаёт масштаб влияния соседних объектов.

Что удобно в kNN и что ограничивает

Плюсы

- ▶ Понятное правило предсказания.
- ▶ Классификация и регрессия.
- ▶ Можно показать соседей для конкретного ответа.

Ограничения

- ▶ Нужно хранить обучающие объекты.
- ▶ Полный поиск: порядка nd операций на запрос.
- ▶ Масштабы и лишние признаки меняют расстояния.

Настройки: признаки, расстояние, k , веса, ядро и окно h .

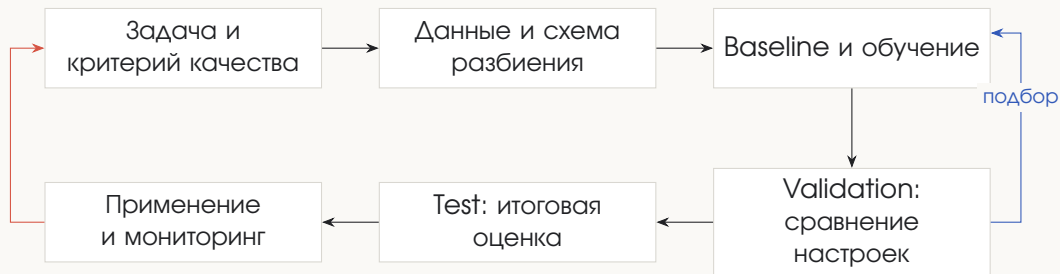
Чтобы сравнивать их честно, нужен общий протокол эксперимента.

2

ML-пайплайн

Постановка задачи, разбиение и выбор модели

Полный цикл ML-задачи



У каждой оценки должна быть роль: выбрать решение или проверить уже выбранное.

Постановка задачи до обучения

Пример: фильтр входящей почты. Решение принимаем при получении письма.

Элемент задачи	Что фиксируем
Объект и target	Одно письмо; $y = 1$ — спам, $y = 0$ — обычное письмо.
Доступные признаки	Текст, отправитель, ссылки — на момент получения.
Действие	Автоматически отправить письмо в спам.
Критерий	Ограничить долю обычных писем среди заблокированных; затем найти как можно больше спама.

Значения метрик разберём в следующем блоке.

Оценка на обучающих данных

У kNN с $k = 1$ обучающий объект может найти самого себя.

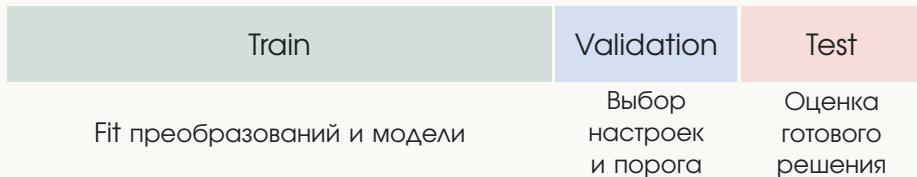
Расстояние 0 $\rightarrow$ своя метка $\rightarrow$ все ответы верны

Для уникальных объектов без конфликтующих меток.

Такая проверка показывает запоминание известных примеров.

Нас интересуют новые объекты, которые модель ещё не видела.

Три части данных



На схеме условные доли 60/20/20. Универсальной пропорции нет.

- ▶ Сначала решаем, какие новые данные модель встретит после запуска.
- ▶ Проверяем, хватает ли объектов редкого класса для оценки.
- ▶ Test не участвует в выборе признаков, модели и порога.

Разбиение под сценарий применения

Что предсказываем	Как делим	Что может зависить оценку
Независимые новые объекты	Случайно; при необходимости со стратификацией	Копии одного объекта с обеих сторон
Данные новых пользователей	По ID пользователя	Один человек и в train, и в test
Будущие события	По времени	Обучение на событиях из будущего

Стратификация сохраняет доли классов, но не устраняет зависимости между объектами.

Десять записей одного человека

Есть 100 пользователей, по 10 записей от каждого.

После запуска модель будет работать с **новыми пользователями**.

Вариант	Разбиение
A	Случайно распределить 1000 записей между train и test.
B	Выбрать 80 пользователей для train и 20 других для test.

Какой вариант соответствует задаче? Что измерит другой вариант?

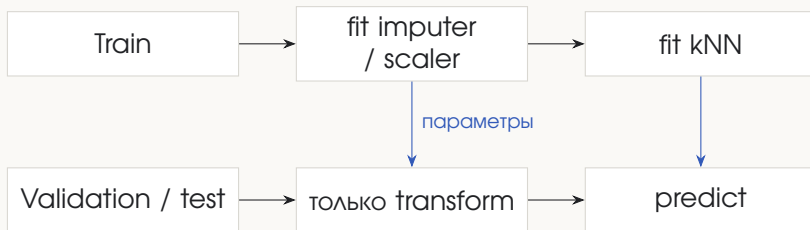
Разбор: единица разбиения

Для новых пользователей выбираем В.

- ▶ Все записи одного пользователя остаются в одной части.
- ▶ В варианте А модель может использовать особенности уже знакомого человека.
- ▶ Такой тест не отвечает на вопрос о качестве на новых пользователях.

А для будущих записей знакомых пользователей нужно отдельно учесть время.

Обучение преобразований



Медианы для пропусков и параметры scaler определяем по train. На новых данных применяем те же преобразования.

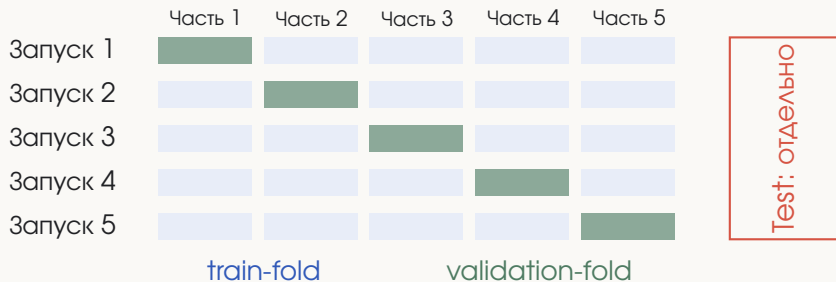
Утечка через признаки

Хотим определить спам в момент получения письма.

Признак	Доступен в этот момент?
Количество ссылок в письме	Да
Репутация отправителя по прошлым письмам	Да, если только по прошлому
Пожаловался ли адресат на это письмо	Нет: это произойдёт позже

Даже правильный train/test split не спасает, если признаки содержат информацию из будущего.

K-fold cross-validation



Для каждой настройки — пять обучений и пять оценок. Сравниваем среднее и разброс.

В каждом запуске преобразования и модель обучаются заново на train-fold.

CV заменяет отдельную validation-часть; test по-прежнему отложен.

Подбор гиперпараметров

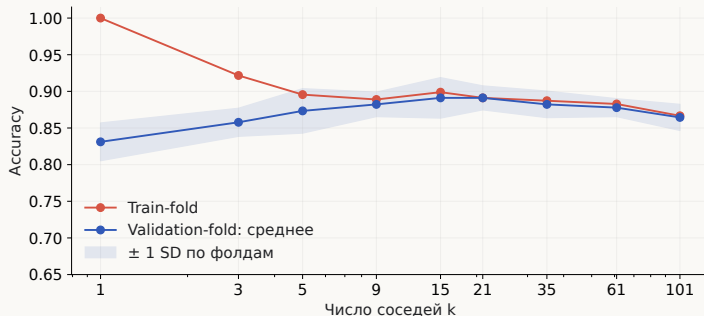
Пример пространства поиска: $k \in \{3, 9, 21\}$, расстояние L_1/L_2 , веса одинаковые / по расстоянию.

$$3 \times 2 \times 2 = 12 \text{ настроек}, \quad 12 \times 5 = 60 \text{ обучений при 5-fold CV.}$$

- ▶ До сравнения задаём основную метрику и допустимые ограничения.
- ▶ Выбираем настройку по validation или CV.
- ▶ Фиксируем выбор и только затем оцениваем на test.

Параметры учатся из train; гиперпараметры задают способ обучения и выбираются по validation.

Train и validation при разных k



Accuracy — доля верных ответов. L_2 , равные веса; 5-fold CV на синтетических данных. При $k = 1$ train выглядит идеально. Настройку выбираем по validation.

Проверка: где утечка данных?

Студент сделал так:

1. Обучил StandardScaler на всех объектах.
2. Заполнил пропуски медианами по всей таблице.
3. Разделил данные на train и test.
4. Сравнил 20 значений k по test accuracy.
5. Опубликовал лучший результат.

Что нужно изменить и в каком порядке?

Разбор: порядок эксперимента

1. Отделить `test` до обучения преобразований.
2. На оставшихся данных организовать `validation` или `CV`.
3. Внутри каждой обучающей части обучать `imputer` → `scaler` → `kNN`.
4. Выбрать k и остальные настройки по `validation` / `CV`.
5. После фиксации решения получить одну итоговую оценку на `test`.

Pipeline помогает удерживать этот порядок внутри `cross-validation`.

3

Метрики качества

Какие ошибки скрываются за одним числом

Accuracy при редком классе

На проверке 100 писем. Модель всегда отвечает: «обычное письмо».



$$\text{Accuracy} = \frac{\text{число верных ответов}}{\text{число всех объектов}}.$$

Какая здесь ассурасу? Сколько спама модель обнаружила?

Одинаковые данные, разные ошибки

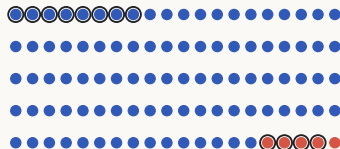
Модель A: accuracy 96%



Найдено спама: 3 из 5

Обычных писем заблокировано: 2

Модель B: accuracy 91%



Найдено спама: 4 из 5

Обычных писем заблокировано: 8

Красный — спам, синий — обычное. Обводка — предсказали «спам».

У ответа «всё обычное» accuracy 95%, найдено 0 из 5.

Accuracy скрывает, какие именно объекты модель пропускает.

Матрица ошибок и метрики

Confusion matrix: модель B, $N = 100$ писем; положительный класс — спам.

Истина	Предсказание	
	Спам	Обычное
Спам	TP = 4	FN = 1
Обычное	FP = 8	TN = 87

TP / TN: верные ответы.

FP: обычное письмо заблокировано.

FN: спам пропущен.

Accuracy

$$\frac{TP + TN}{N} = \frac{4 + 87}{100} = 91\%$$

Precision

$$\frac{TP}{TP + FP} = \frac{4}{12} \approx 33.3\%$$

Recall / TPR

$$\frac{TP}{TP + FN} = \frac{4}{5} = 80\%$$

F1

$$\frac{2TP}{2TP + FP + FN} = \frac{8}{17} \approx 0.47$$

FPR

$$\frac{FP}{FP + TN} = \frac{8}{95} \approx 8.4\%$$

Specificity / TNR

$$\frac{TN}{TN + FP} = \frac{87}{95} \approx 91.6\%$$

Precision и recall: разные знаменатели

Отобрали 12 писем



$$P = \frac{TP}{TP + FP} = \frac{4}{12} \approx 33\%.$$

Доля настоящего спама среди **выбранных** писем.

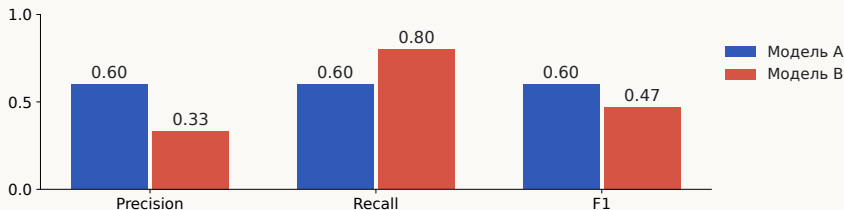
Всего 5 спам-писем



$$R = \frac{TP}{TP + FN} = \frac{4}{5} = 80\%.$$

Доля найденных среди **всех** спам-писем.

F1: совместная оценка precision и recall



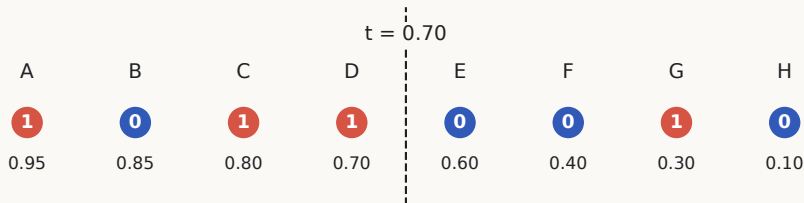
$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}$$

Для B: $F_1 = 8/17 \approx 0.47$. У A: $F_1 = 0.60$.

F1 учитывает обе ошибки, но не задаёт их цену для задачи.

Score и порог решения

Теперь 8 объектов: четыре положительных и четыре отрицательных.

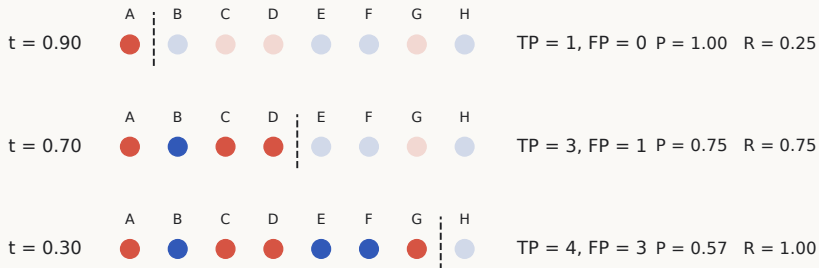


$$\hat{y} = 1[s(x) \geq t].$$

В kNN score можно получить как долю голосов класса 1.

При $t = 0.70$ посчитайте TP, FP, FN и TN.

Что меняется при снижении порога



При $t = 0.70$: $TP = 3$, $FP = 1$, $FN = 1$, $TN = 3$.

Порог ниже — больше объектов отбираем, больше положительных находим.

Precision при этом может как снижаться, так и расти.

Один порог — одна точка ROC

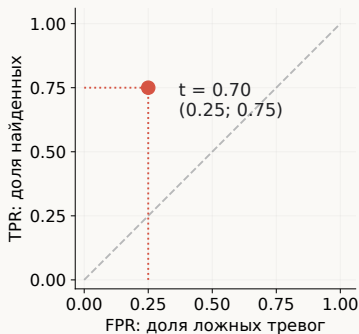
Порог $t = 0.70$:

нашли 3 из 4 положительных;

ошиблись на 1 из 4 отрицательных.

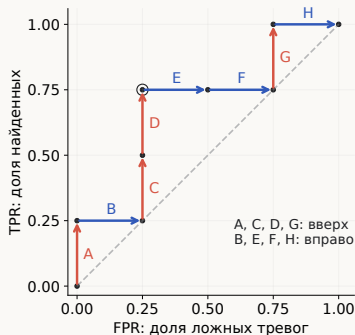
$$\text{TPR} = \frac{TP}{TP + FN} = \frac{3}{4}.$$

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{1}{4}.$$



FPR — доля ошибок среди всех отрицательных объектов. Это другой знаменатель, чем у precision.

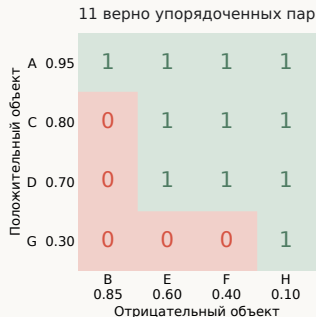
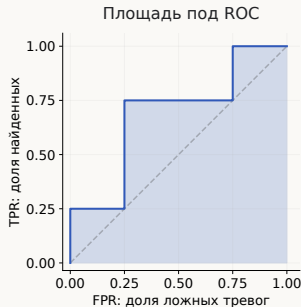
Построение ROC-кривой



Порядок по score: A, B, C, D, E, F, G, H.

Положительный: шаг вверх на $1/4$. Отрицательный: вправо на $1/4$.

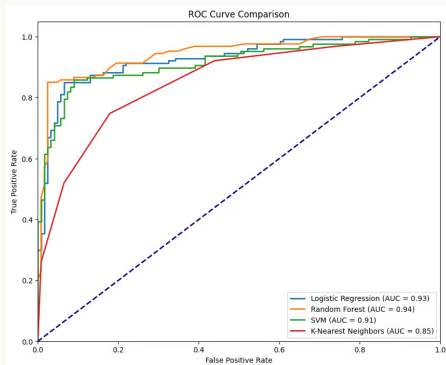
ROC-AUC: площадь и правильные пары



$$AUC = \frac{\text{пар, где score положительного выше}}{\text{всех пар}} = \frac{11}{16} = 0.6875.$$

AUC = 1: идеальный порядок. 0.5: случайный порядок в среднем. 0: обратный.

Сравнение ROC-кривых



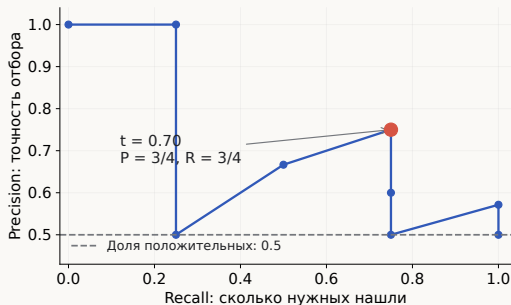
Сравниваем модели на **одной** проверочной выборке.

В этом примере самая большая площадь у Random Forest: 0.94.

Если кривые пересекаются, преимущество зависит от допустимой FPR.

ROC-AUC суммирует всю кривую. Рабочий порог выбираем отдельно.

PR-кривая: точность отбора и полнота

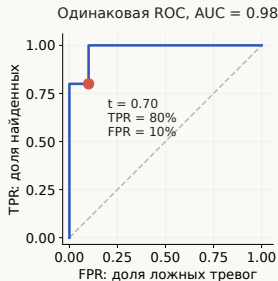


Порог 0.70: нашли 3 из 4 нужных, и 3 из 4 выбранных — нужные.

Средняя точность по ранжированию: average precision (AP) ≈ 0.747 .

ROC-AUC при редком классе

Две выборки по 10 000 объектов. Кривые ROC совпадают.

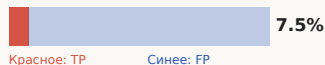


Состав срабатываний при одном пороге

Положительных 50%: 4000 TP + 500 FP



Положительных 1%: 80 TP + 990 FP



При доле 1%: из 1 070 срабатываний только 80 верны.

Для редкого класса проверяем precision и рабочий порог.

Метрика и последствия ошибок

Действие и ошибка	Что оценивать
Спам: не потерять обычное письмо	Recall при минимуме precision
Скрининг: не пропустить нужный случай	Recall и нагрузку от FP
Fraud: отобрать редкие нарушения	PR / AP и рабочую precision
Скоринг: ранжировать риск	ROC-AUC и выбранный порог
Токсичность: учитывать оба типа ошибок	P/R; F1 при таком критерии
Цифры: равноценные ошибки	Accuracy и baseline

Название задачи само по себе не определяет метрику. Важно действие.

Проверка: выбор рабочего порога

Автоматически удаляем спам. По условию задачи precision должен быть не ниже 90%.

Порог на validation	Precision	Recall
t_1	.96	.35
t_2	.91	.70
t_3	.78	.88

Какой порог выбрать, если среди допустимых решений хотим найти больше спама? Где проверить результат после выбора?

Разбор: метрика и ограничение

Выбираем t_2 : precision = 91%, recall = 70%.

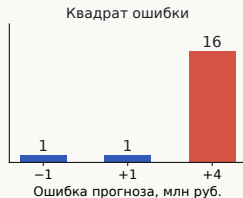
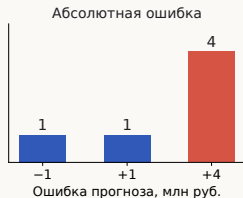
- ▶ t_3 нарушает ограничение.
- ▶ t_1 допустим, но находит меньше спама.
- ▶ После фиксации порога оцениваем решение на test.

Для модерации с ручной проверкой ограничением может стать число проверок в день. Тогда критерий выбора будет другим.

Условие на validation не гарантирует такую же precision на будущих данных.

Метрики регрессии

Ошибки трёх прогнозов цены: -1 , $+1$, $+4$ млн руб.



$$\text{MAE} = \frac{1 + 1 + 4}{3} = 2, \quad \text{RMSE} = \sqrt{\frac{1 + 1 + 16}{3}} \approx 2.45.$$

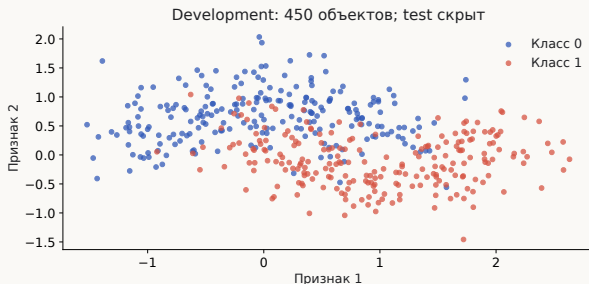
Обе метрики в млн руб. RMSE сильнее реагирует на большую ошибку.

4

Законченный эксперимент

От baseline до test и применения модели

Данные и протокол эксперимента



600 синтетических объектов; 2 признака; два класса поровну. Ошибки считаем равноценными.

До обучения: метрика accuracy; 450 объектов для разработки, 150 — test.

На 450 объектах проводим 5-fold CV: 360 train и 90 validation в каждом запуске.

Pipeline внутри подбора настроек

```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
grid = {"knn__n_neighbors": [3, 9, 21],
        "knn__p": [1, 2],
        "knn__weights": ["uniform", "distance"]}
search = GridSearchCV(pipe, grid, cv=cv,
                      scoring="accuracy")
search.fit(X_dev, y_dev)
```

12 конфигураций × 5 фолдов = 60 обучений.

Scaler заново обучается внутри каждого train-fold.

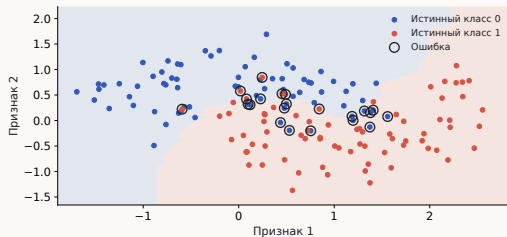
Выбор по CV, проверка на test

Этап	Результат
Выбрано по CV	$k = 21, L_1$, веса по расстоянию
Accuracy на CV	0.896; SD по фолдам 0.018
Затем	Переобучение выбранного pipeline на всех 450 объектах
Accuracy на test	0.853 (128 из 150)
Baseline на test	0.500: всегда один класс

Test открываем после выбора конфигурации. Это один учебный эксперимент; небольшие различия требуют отдельной проверки.

Разбор ошибок выбранной модели

На test:



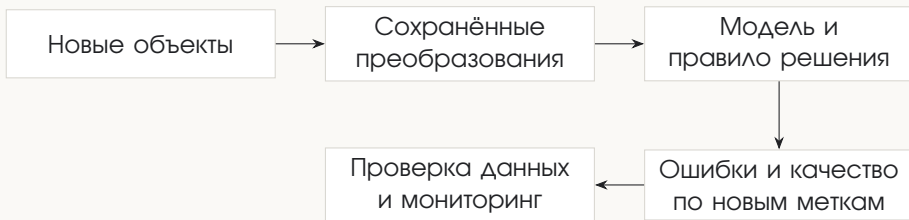
Истина	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	60	15
$y = 1$	7	68

Ошибок: 22 из 150.

Чёрным обведены неверные ответы; фон — решение модели.

Разбор помогает найти идеи для следующей итерации. Если меняем решение по test, для новой итоговой оценки нужен независимый набор.

Применение и мониторинг



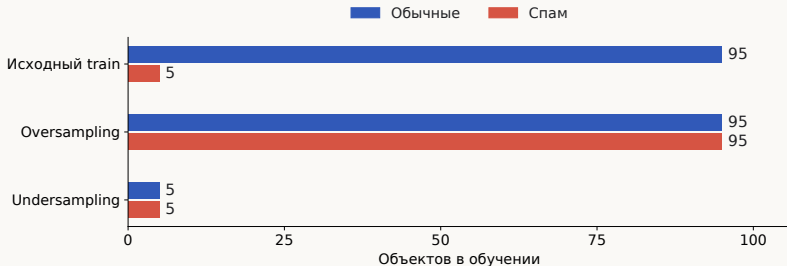
- ▶ Сохраняем весь pipeline вместе с моделью.
- ▶ Следим за входными данными, долей решений, задержкой и качеством.
- ▶ При изменениях повторяем эксперимент; новые метки могут приходить с задержкой.

5

Дисбаланс классов

Изменение обучающей выборки и SMOTE

Как меняем обучающую выборку



Oversampling: повторяем редкие объекты.

Undersampling: удаляем часть частого класса.

Порог меняет решения модели. Ресемплинг меняет данные для обучения.

SMOTE: синтетический объект



Для x_i случайно выбираем x_j среди k_{SMOTE} ближайших объектов того же класса.

$$x_{new} = x_i + \alpha(x_j - x_i), \quad \alpha \sim U(0, 1).$$

При $x_i = (1, 1)$, $x_j = (5, 3)$, $\alpha = 0.25$:

$$x_{new} = (2, 1.5).$$

Синтетическая точка наследует метку редкого класса.

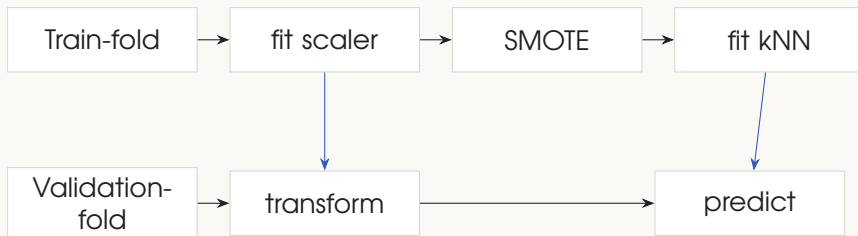
Ограничения SMOTE



- ▶ Отрезок может пересечь область другого класса.
- ▶ Точки около выбросов усиливают ошибочную структуру.
- ▶ Обычный SMOTE предполагает числовые признаки и осмысленную интерполяцию.

SMOTE не добавляет независимых наблюдений и не гарантирует улучшения качества.

SMOTE внутри обучающего фолда



Validation и test сохраняют исходные объекты и доли классов.

Реализация: `imblearn.pipeline.Pipeline`. На predict шаг ресемплинга пропускается.

6

Поиск и RAG

Соседи как результат поиска

Поиск соседей в RAG

В классификации соседи голосуют за класс. В поиске возвращаем сами найденные фрагменты.



Векторы запроса и документов должны быть сопоставимы. Близость можно считать по cosine.

Оценка поиска



$$\text{Precision@2} = \frac{1}{2}, \quad \text{Recall@2} = \frac{1}{3}.$$

В первых двух результатах один полезный. Всего полезных в корпусе три.

Качество ответа проверяем отдельно: верен ли он и подтверждается ли источниками.

Теоретический минимум

Тема	Нужно уметь
kNN	Объяснить соседей, веса, ядро и окно; классификацию и регрессию.
Пайплайн	Задать target, признаки, baseline и критерий качества.
Разбиение	Различить train / validation / test; выбрать схему split.
CV и утечки	Fit внутри train-fold; подбор настроек без test.
Метрики	Объяснить провал accuracy; считать P/R/F1, MAE/RMSE.
Порог и ранжирование	Построить ROC по score; объяснить AUC и роль PR.

SMOTE — только на train. В RAG отдельно оцениваем поиск и ответ.

Ко(т)нец презентации



От постановки задачи до
проверки на новых данных.

Вопросы?